



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации информационных
технологий

Отчет по практическим работам №5-8

по дисциплине «Системная и программная инженерия»

Выполнили:
Студенты группы **ИКБО-66-23**

Гордильо А.
Смирнов А.Ю.
Ковалёв А. Э.
Тищенко И. А.

Проверил:

Преподаватель Левандровский А.М.

МОСКВА 2026 г.

Содержание

1. СОЗДАНИЕ СТРУКТУРНОЙ ДИАГРАММЫ СИСТЕМЫ	3
1.1. Структурная диаграмма	3
1.2. Построение диаграммы процессов в нотации BPMN	4
1.3. Ход работы	5
2. СОЗДАНИЕ ИНФОРМАЦИОННОЙ ДИАГРАММЫ СИСТЕМЫ И ЛОГИЧЕСКОЙ МОДЕЛИ БАЗЫ ДАННЫХ.....	10
2.1. Диаграмма в нотации DFD.....	10
2.2. Ход работы	10
3. АРХИТЕКТУРА СИСТЕМЫ	12
3.1. Информационное взаимодействие компонентов системы	12
3.2. Архитектура системы.....	14
3.3. Архитектурная диаграмма разработки	17
3.4. Матрица требований.....	17
4. ТЕХНИЧЕСКОЕ ЗАДАНИЕ	23
4.1. Обоснование выбора ГОСТ для разработки технического задания	23
4.2. Техническое задание на создание автоматизированной системы "TicketFlow" (по ГОСТ 34.602-2020)	25
Заключение.....	28

1. СОЗДАНИЕ СТРУКТУРНОЙ ДИАГРАММЫ СИСТЕМЫ

1.1. Структурная диаграмма

Главной целью структурных диаграмм является наглядное графическое отображение внутреннего состава различных совокупностей и систем путём демонстрации соотношения между их отдельными частями или элементами.

Структурная диаграмма – это инструмент модульного проектирования сверху вниз. Она строится из прямоугольников, каждый из которых обозначает отдельный модуль системы, и линий, соединяющих эти модули между собой. Линии на диаграмме показывают связи между модулями, а также отношения подчинённости или владения. По принципу построения структурная диаграмма похожа на организационную диаграмму, однако используется она именно для представления структуры программной системы или её отдельных частей

Диаграмма классов – это одна из ключевых структурных диаграмм языка моделирования UML. Она предназначена для графического представления статической структуры системы, отображая иерархию классов, их атрибуты (поля), методы (операции), интерфейсы, а также различные типы взаимосвязей между ними.

Диаграмма объектов – это структурная диаграмма языка моделирования UML, которая показывает набор конкретных объектов системы и связи между ними на определённый момент времени.

1.2. Построение диаграммы процессов в нотации BPMN

Нотации – это специальные графические модели, предназначенные для фиксации, анализа и оптимизации бизнес-процессов. В отличие от обычных текстовых описаний, нотации позволяют компактно и наглядно представить весь ход процесса – от его начала до завершения. Благодаря визуальному формату значительно проще воспринимать последовательность действий, выявлять логику алгоритма и видеть общую структуру процесса.

Нотации применяются, чтобы сотрудники могли быстро понять и запомнить порядок выполнения задач. Например, схема наглядно показывает, как обрабатывать заявку на поставку партии товара. Руководителю такая схема помогает выявить проблемные или избыточные этапы и сотрудников, после чего внести необходимые корректировки. Часто это позволяет ускорить работу компании или снизить её затраты. BPMN (Business Process Model and Notation) — одна из самых популярных нотаций для моделирования бизнес-процессов. Она стандартизирована и поддерживается большинством современных инструментов.

Данная нотация позволяет отразить в модели:

- последовательность выполнения операций (алгоритм процесса);
- роли и зоны ответственности (пулы и дорожки);
- события (начало, конец, таймеры, сообщения);
- ветвления и параллельное выполнение задач (шлюзы).

Модель BPMN строится слева направо. Процесс начинается со стартового события, после которого следуют задачи, шлюзы, промежуточные события и завершается конечным событием. Дорожки (lanes) внутри пулов (pools) служат для распределения ответственности между разными исполнителями. Например, это могут быть дорожки «Пользователь», «Сервер API», «Платежный шлюз». Чем выше расположена дорожка, тем более общая роль она описывает, чем ниже — тем более конкретная.

Основное преимущество BPMN заключается в высокой наглядности и

понятности для широкого круга сотрудников — от бизнес-аналитиков до разработчиков. Нотация позволяет моделировать даже сложные процессы, включая параллельные ветвления, тайм-ауты, обработку ошибок и взаимодействие с внешними системами. Благодаря стандартизации BPMN-диаграммы легко переносить между различными инструментами, а также автоматически исполнять их в специальных BPMN-движках.

1.3. Ход работы

Диаграмма классов на Рисунке 1.1 описывает структуру мобильного приложения, предназначенного для планирования и управления встречами, координации участников и отслеживания расписания. Центральным элементом системы является класс **User**, представляющий пользователя и содержащий идентификаторы, контактные данные (имя, email, пароль) и методы взаимодействия с системой, такие как регистрация, вход в систему и редактирование профиля.

С классом **User** напрямую связаны классы **Calendar**, **MeetingRequest**, **Feedback** и **Notification**.

- **Calendar** хранит расписание пользователя, включая список его встреч. Методы класса позволяют добавлять встречу, удалять её и получать полный список запланированных событий.
- **MeetingRequest** отражает запросы на участие в встречах: пользователь может создавать, отменять или подтверждать запрос на встречу.
- **Feedback** позволяет пользователю оценить встречу, оставляя рейтинг и комментарий, что помогает организаторам улучшать взаимодействие и качество встреч.
- **Notification** отвечает за отправку уведомлений пользователю (например, напоминание о встрече, изменение времени или подтверждение участия).

Класс **Meeting** описывает саму встречу: название, дату и время, описание. Пользователь может создавать встречи, обновлять их или отменять. Каждая встреча проводится в **Location**, которая содержит название и адрес.

Локация имеет множество TimeSlot — временных слотов, доступных для бронирования. Методы reserve() и free() позволяют управлять доступностью слотов для встреч.

Класс Invitation отражает приглашения на встречу: каждый пользователь получает приглашение, которое он может принять или отклонить. Связь между Invitation и User указывает на того, кому направлено приглашение, а связь с Meeting — к какой встрече оно относится.

Класс System является управляющим компонентом приложения: он хранит списки всех пользователей и встреч, обеспечивает регистрацию пользователей, вход в систему и создание встреч.

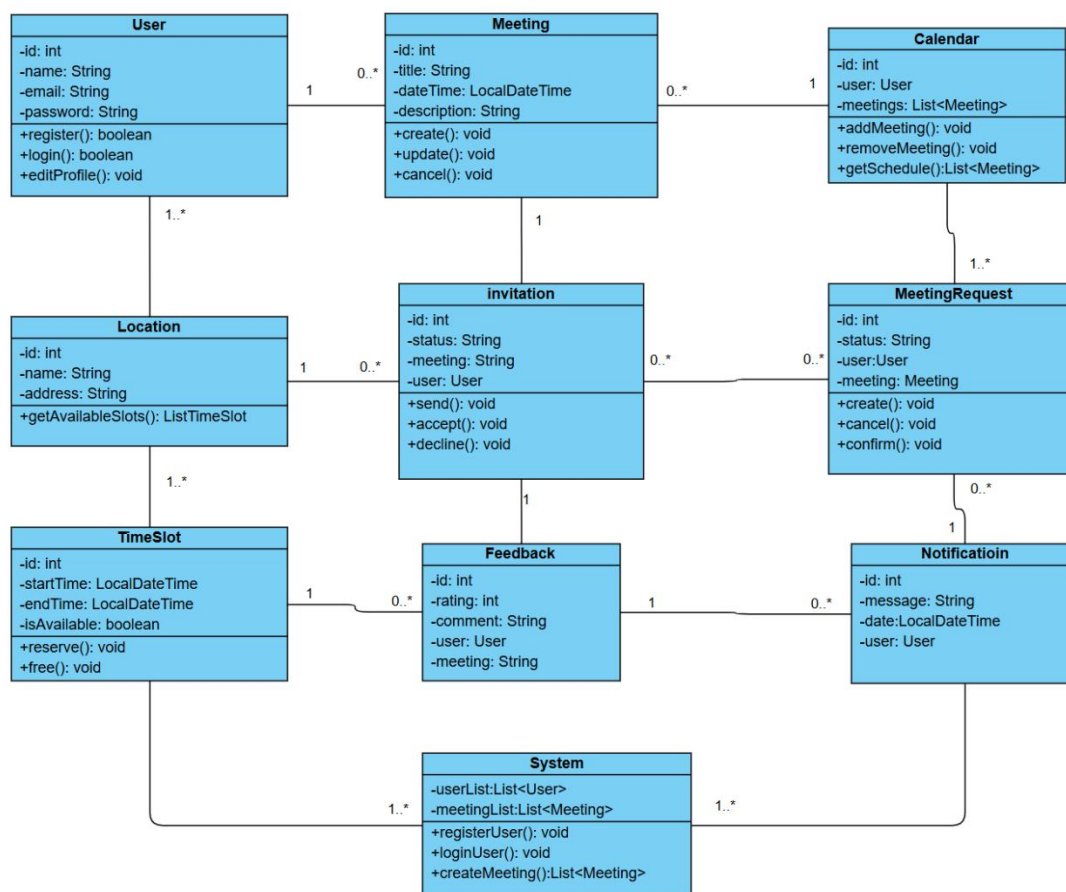


Рисунок 1.1 – Диаграмма классов

Диаграмма объектов на Рисунке 1.2 иллюстрирует конкретное состояние системы в момент времени, когда пользователь уже вошёл в приложение, выбрал встречу (проектное совещание) и оформляет заказ на её подтверждение/участие.

Объект meeting1: Meeting

Представляет конкретную встречу (например, «Project Update»). Содержит дату, время, длительность и заголовок. Связан с локацией и заметками.

Объект location1: Location

Место проведения встречи. Содержит название площадки («Office») и адрес.

Объекты заметок (note1: Note)

Отражают содержание встречи: созданы пользователем (createdby->user1), связаны с конкретной встречей (meeting->meeting1), содержат описание («Discuss project»).

Объект order1: Order

Отражает оформленный заказ пользователя: содержит статус («confirmed»), связан с пользователем user1

Объект notification1: Notification

Отправлен пользователю: напоминание о скором начале встречи («Meeting starts soon»), содержит email и пароль (вероятно, для учётной записи, с которой отправлено)

Объект system1: System

Содержит: список пользователей (включая user1), список встреч (meeting1), заказы (order1), а также список сообщений (chat1)

Объект admin1: Admin

Представляет администратора системы, который управляет её работой через system1:

- Идентификатор (id=1)
- Адрес электронной почты
- Пароль для доступа к системе

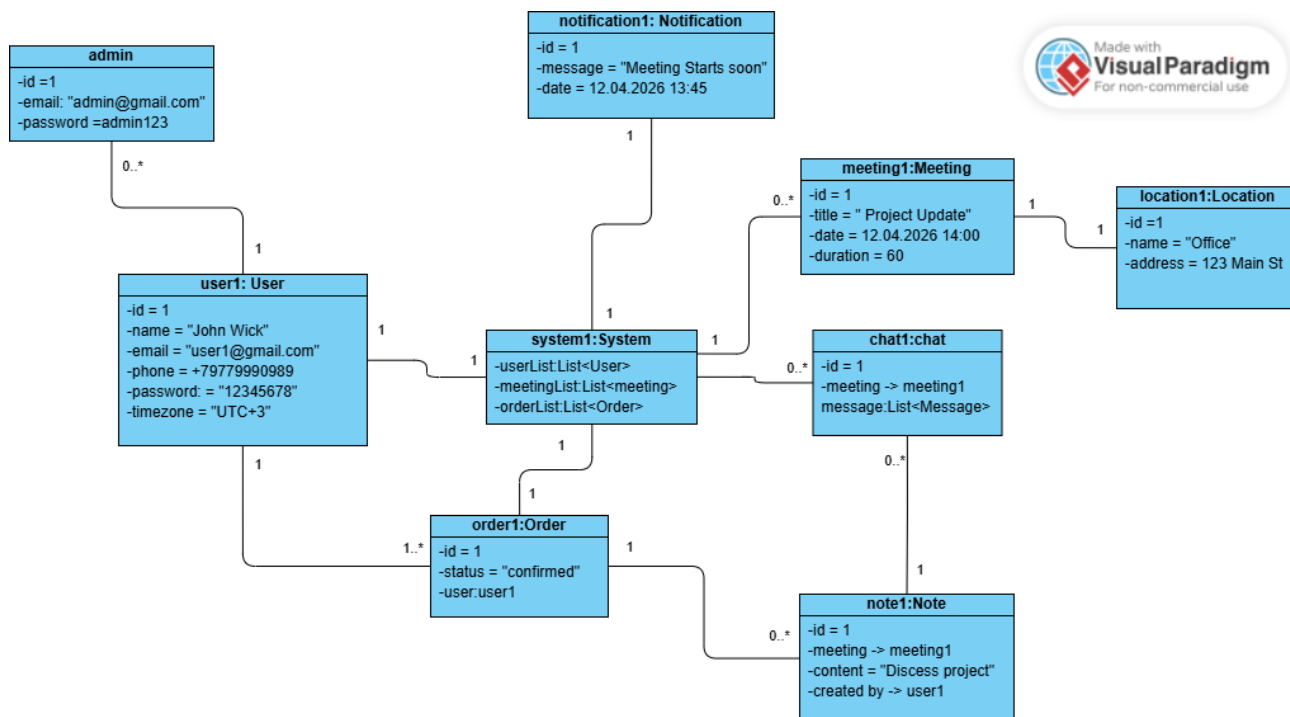


Рисунок 1.2 – Диаграмма объектов

Диаграмма BPMN описывает процесс создания встречи пользователем в мобильном приложении. Процесс начинается с открытия приложения. Пользователь инициирует создание встречи. Система проверяет, свободны ли выбранные дата и время. Если дата или время заняты, пользователь возвращается к выбору новой даты и времени. Если дата и время свободны, система проверяет, принято ли приглашение. Если приглашение не принято, процесс завершается. Если приглашение принято, встреча считается созданной и подтверждённой. После этого система отправляет уведомления всем участникам встречи. Процесс завершается.

Система проверяет доступность выбранных даты и времени. В случае занятости пользователь возвращается к выбору даты и времени, иначе система проверяет, принято ли приглашение. Если приглашение не принято, процесс завершается. Если приглашение принято, встреча считается созданной и подтверждённой. Далее система отправляет уведомления всем участникам встречи.

После подтверждения встречи происходит проверка принятия

приглашения. В случае отказа процесс завершается, в противном случае встреча считается созданной и подтвержденной. Далее система отправляет уведомления всем участникам встречи. Процесс завершается отправкой уведомлений.

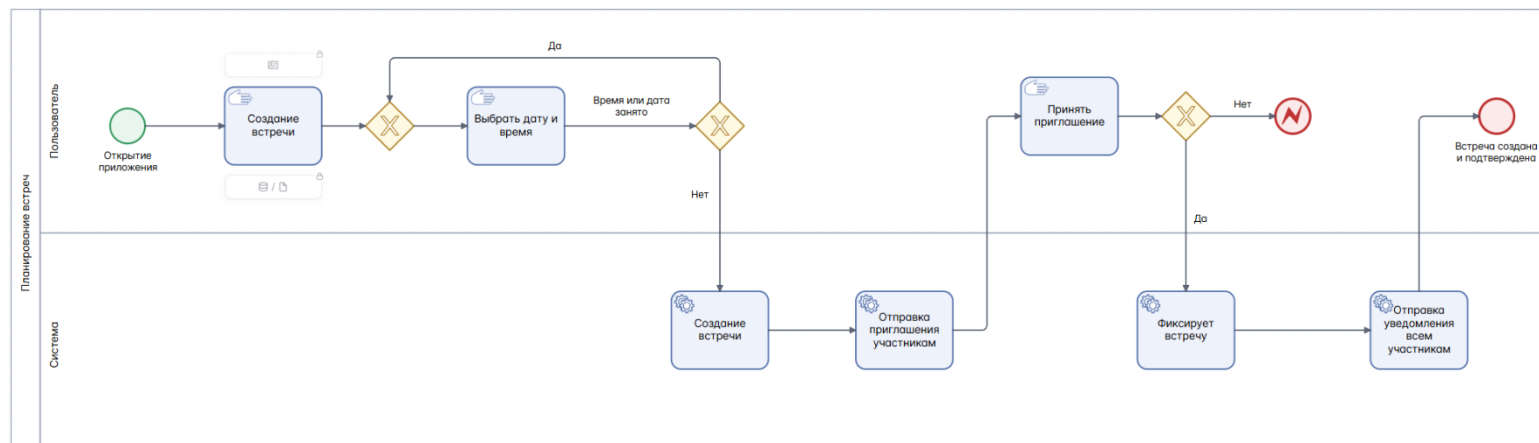


Рисунок 1.3 – BPMN Диаграмма

2. СОЗДАНИЕ ИНФОРМАЦИОННОЙ ДИАГРАММЫ СИСТЕМЫ И ЛОГИЧЕСКОЙ МОДЕЛИ БАЗЫ ДАННЫХ

2.1. Диаграмма в нотации DFD

DFD — общепринятое сокращение от англ. data flow diagrams — диаграммы потоков данных. Так называется методология графического структурного анализа, описывающая внешние по отношению к системе, источники и адресаты данных, логические функции, потоки данных и хранилища данных, к которым осуществляется доступ. Диаграмма потоков данных (data flow diagram, DFD) — один из основных инструментов структурного анализа и проектирования информационных систем, существовавших до широкого распространения UML.

2.2. Ход работы

На рисунках 2.1 и 2.2 показаны диаграммы в нотации DFD, описывающие потоки данных в ходе работы системы.

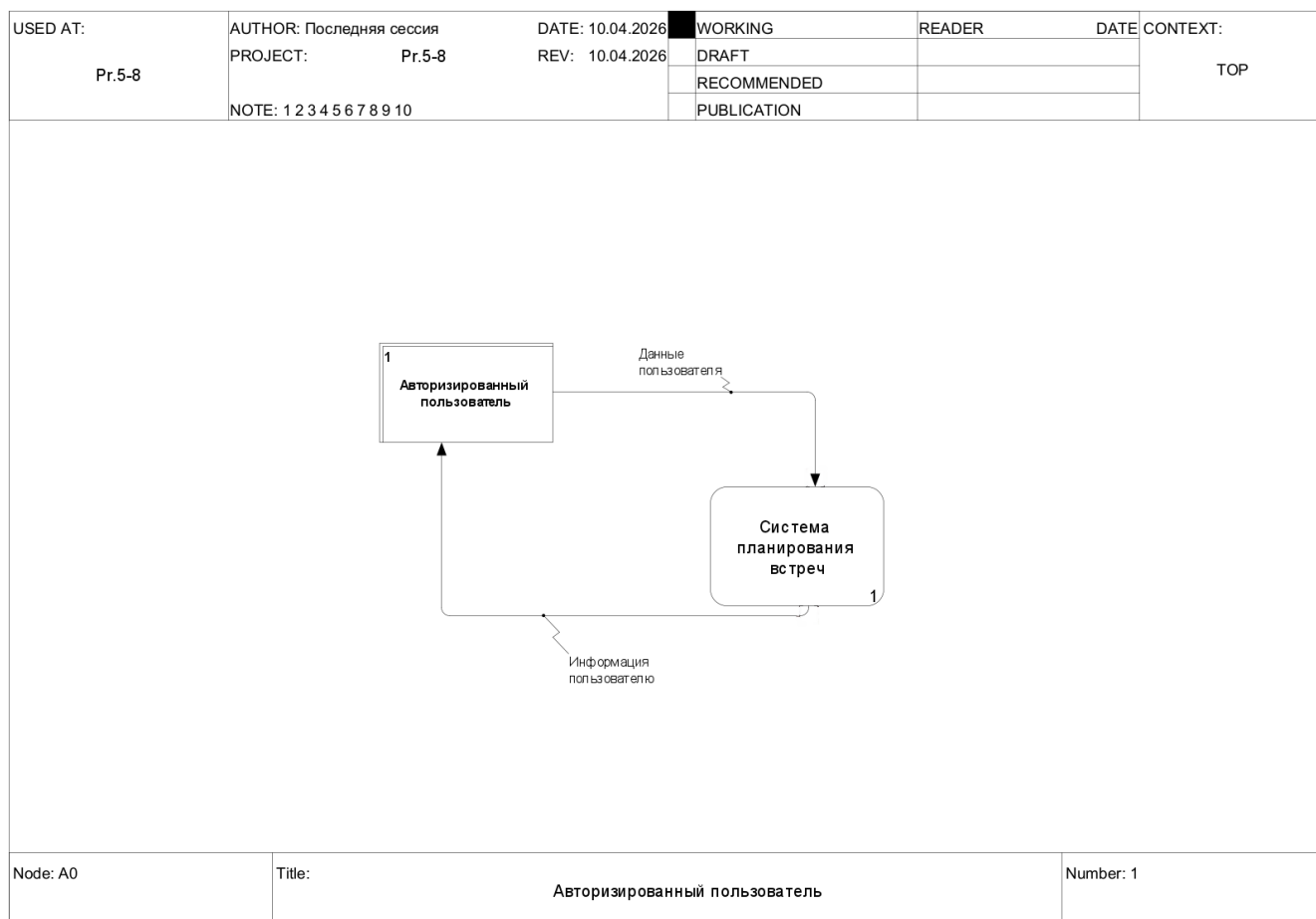


Рисунок 2.1 – Контекстная диаграмма

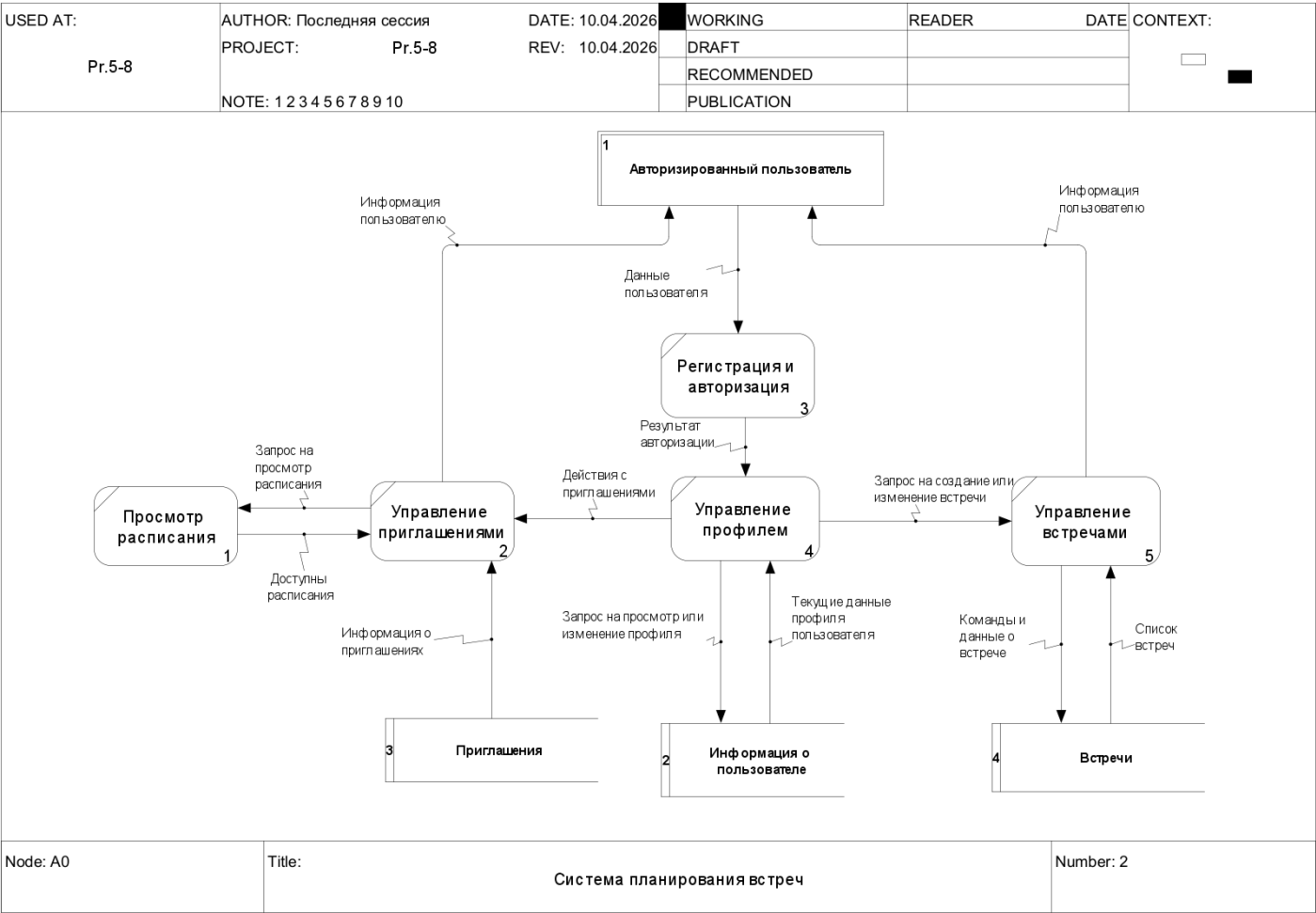


Рисунок 2.2 – Декомпозиция контекстной диаграммы

3. АРХИТЕКТУРА СИСТЕМЫ

3.1. Информационное взаимодействие компонентов системы

Первый уровень — **уровень представления** — реализован в виде мобильного приложения для платформы Android. Данный уровень обеспечивает взаимодействие пользователя с системой и отображение информации в удобной форме. На этом уровне располагаются экраны регистрации и авторизации, просмотра и редактирования профиля, создания встречи, просмотра приглашений и просмотра личного расписания. Пользователь через мобильное приложение вводит необходимые данные, выбирает дату и время встречи, список участников, а также просматривает ответы на приглашения и своё расписание.

На уровне представления также выполняется первичная обработка пользовательских действий: проверка корректности заполнения обязательных полей, валидация email, контроль выбора временного слота только по целым часам, а также проверка того, что встреча создаётся как минимум с одним приглашённым участником. После прохождения первичной проверки мобильное приложение формирует HTTP-запросы и передаёт их на серверную часть через REST API. В клиентской части проекта используется многослойная структура Data/Domain/Presentation, а работа с сетью и локальным хранением токенов описана через Retrofit и DataStore.

Второй уровень — **уровень бизнес-логики** — реализован в виде серверного приложения на базе Spring Boot. Данный уровень является центральным компонентом системы и отвечает за обработку запросов от мобильного клиента, аутентификацию и авторизацию пользователей, управление профилями, создание встреч, проверку конфликтов расписания,

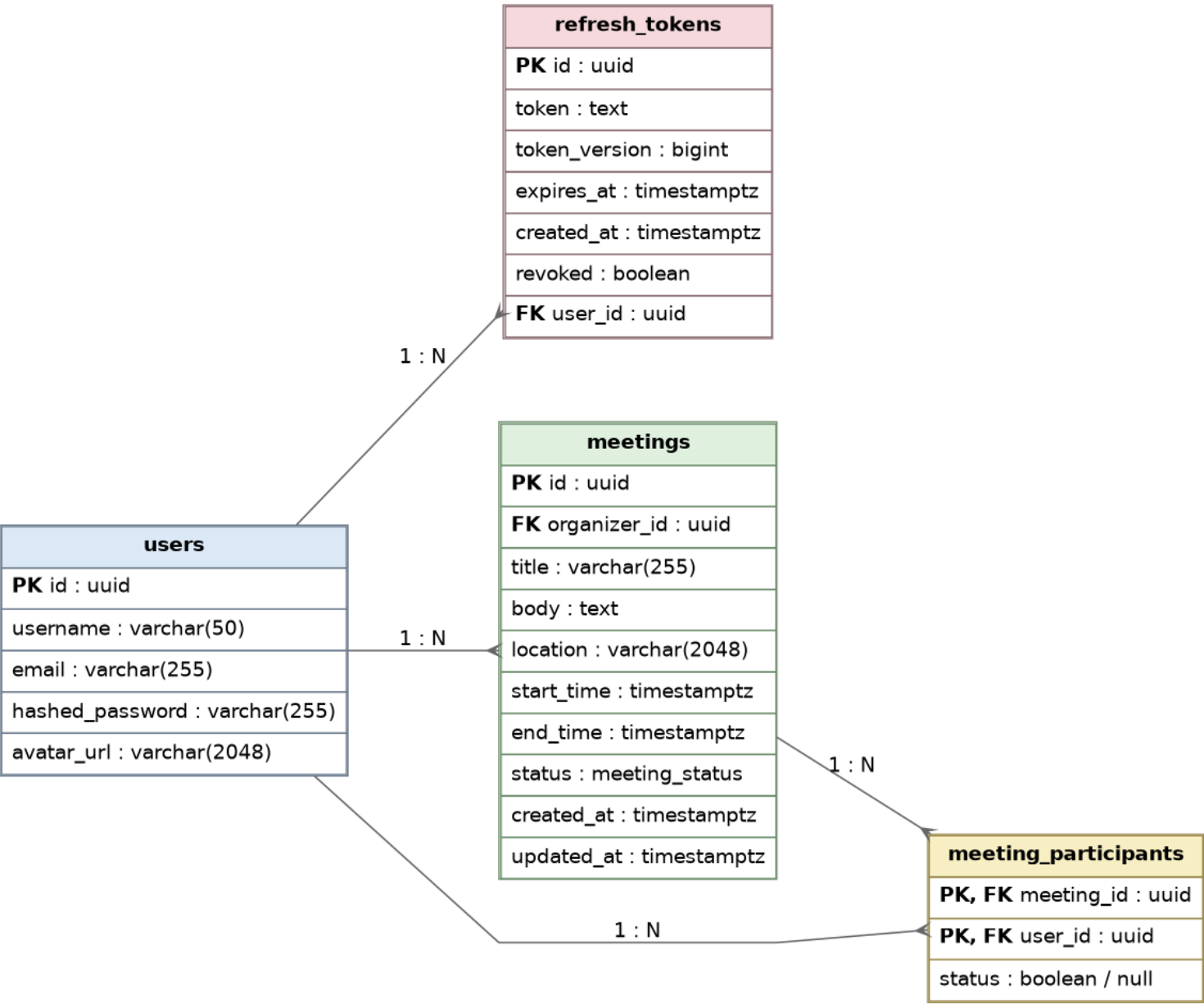
обработку приглашений и формирование ответов клиентскому приложению. На серверной стороне логика организована по слоям controller, service, repository, model, dto и security, что обеспечивает разделение ответственности между компонентами.

При создании встречи сервер получает от клиента данные о теме, описании, месте, времени начала и окончания, а также о списке приглашённых сотрудников. После этого сервер проверяет корректность данных, соблюдение правила часовых слотов и отсутствие пересечений встреч у организатора и приглашённых участников. При успешной проверке сервер сохраняет встречу в базе данных и создаёт записи о приглашениях участников.

Третий уровень — **уровень данных** — предназначен для хранения и управления информацией, используемой системой. В качестве СУБД используется PostgreSQL или H2. На данном уровне хранятся сведения о пользователях, refresh-токенах, встречах и участниках встреч. Все операции с данными выполняются только через серверное приложение, что исключает прямой доступ клиента к базе данных и повышает безопасность системы. Использование Spring Data JPA и Liquibase обеспечивает удобную работу с данными и управляемое изменение структуры базы.

Взаимодействие компонентов системы осуществляется последовательно. Пользователь через мобильное приложение выполняет вход в систему или регистрируется, после чего получает access и refresh токены. Далее пользователь может отправлять запросы на получение профиля, создание встречи, просмотр приглашений и расписания. Сервер принимает запрос, проверяет токен пользователя, выполняет необходимую бизнес-логику и при необходимости обращается к базе данных. После завершения обработки сервер формирует ответ и возвращает его мобильному приложению, которое отображает результат пользователю. Таким образом, система построена по принципу строгого разграничения уровней, что упрощает сопровождение,

3.2. Логическая модель базы данных



4.

Рисунок 3.1 - Логическая модель базы данных

4.1. Архитектура системы

Общее описание архитектуры

Разрабатываемая система представляет собой мобильное приложение для планирования встреч сотрудников внутри компании, реализованное на основе клиент-серверной модели и построенное с использованием трёхуровневой архитектуры.

Система включает следующие основные компоненты:

- мобильное приложение (клиентская часть);
- серверную часть (REST API);
- базу данных.

Архитектура системы построена по принципу разделения ответственности между уровнями. Это позволяет повысить гибкость системы, упростить сопровождение и обеспечить возможность независимого развития клиентской и серверной частей.

Программные решения и обоснование выбора

Клиентская часть

Язык программирования — Kotlin.

Клиентское приложение разрабатывается для платформы Android и построено с использованием Clean Architecture. В проекте выделены слои Domain, Data и Presentation, а для пользовательского интерфейса применяется Jetpack Compose. Для работы с сетью используется Retrofit, для внедрения зависимостей — Hilt, для локального хранения токенов — DataStore. Такой подход обеспечивает хорошую тестируемость, разделение ответственности и удобство сопровождения приложения.

Серверная часть

Серверное приложение реализовано на базе Spring Boot. Для

разработки REST API используется spring-boot-starter-web, для работы с базой данных — spring-boot-starter-data-jpa, для обеспечения безопасности — spring-boot-starter-security, для миграций базы данных — Liquibase, а для работы с JWT — библиотека JWT. Также в проекте используется OpenAPI для документирования API и MapStruct для преобразования DTO и сущностей. Такая архитектура обеспечивает структурированную организацию backend-части и удобство масштабирования.

Аутентификация и авторизация

В системе используется механизм аутентификации на основе JWT. После успешного входа пользователь получает access token и refresh token. Access token применяется для доступа к защищённым ресурсам, а refresh token — для обновления сессии без повторного ввода логина и пароля. Хранение и обработка токенов предусмотрены как на клиентской, так и на серверной стороне.

База данных

В качестве СУБД используется PostgreSQL. Реляционная модель данных хорошо подходит для хранения пользователей, встреч и участников встреч, так как позволяет задавать связи между сущностями, обеспечивать целостность данных и выполнять транзакционные операции. Поддержка JPA упрощает взаимодействие серверной части с базой данных.

Протоколы и форматы обмена данными

Для передачи данных между клиентом и сервером используется протокол HTTP/HTTPS. Формат обмена данными — JSON. Данное решение является стандартным для мобильных клиент-серверных приложений и обеспечивает простоту интеграции, читаемость структуры запросов и ответов, а также независимость клиента и сервера друг от друга.

Преимущества выбранной архитектуры

1. Модульность системы

Каждый уровень системы выполняет строго определённые функции.

2. Масштабируемость

Клиентская и серверная части могут развиваться независимо.

3. Безопасность

Использование JWT, Spring Security и разграничения доступа к данным повышает защищённость системы.

4. Удобство сопровождения

Разделение backend на controller, service, repository, dto и security, а frontend — на Data/Domain/Presentation, упрощает поддержку кода.

5. Надёжность работы

Проверка конфликтов встреч, валидация входных данных и централизованная обработка запросов обеспечивают корректную работу системы.

4.2. Архитектурная диаграмма разработки

На рисунке представлена архитектурная диаграмма разрабатываемой системы. Система построена на основе трехуровневой архитектуры и включает клиентский уровень (мобильное приложение), серверный уровень (backend на базе Ktor) и уровень данных (реляционная база данных).

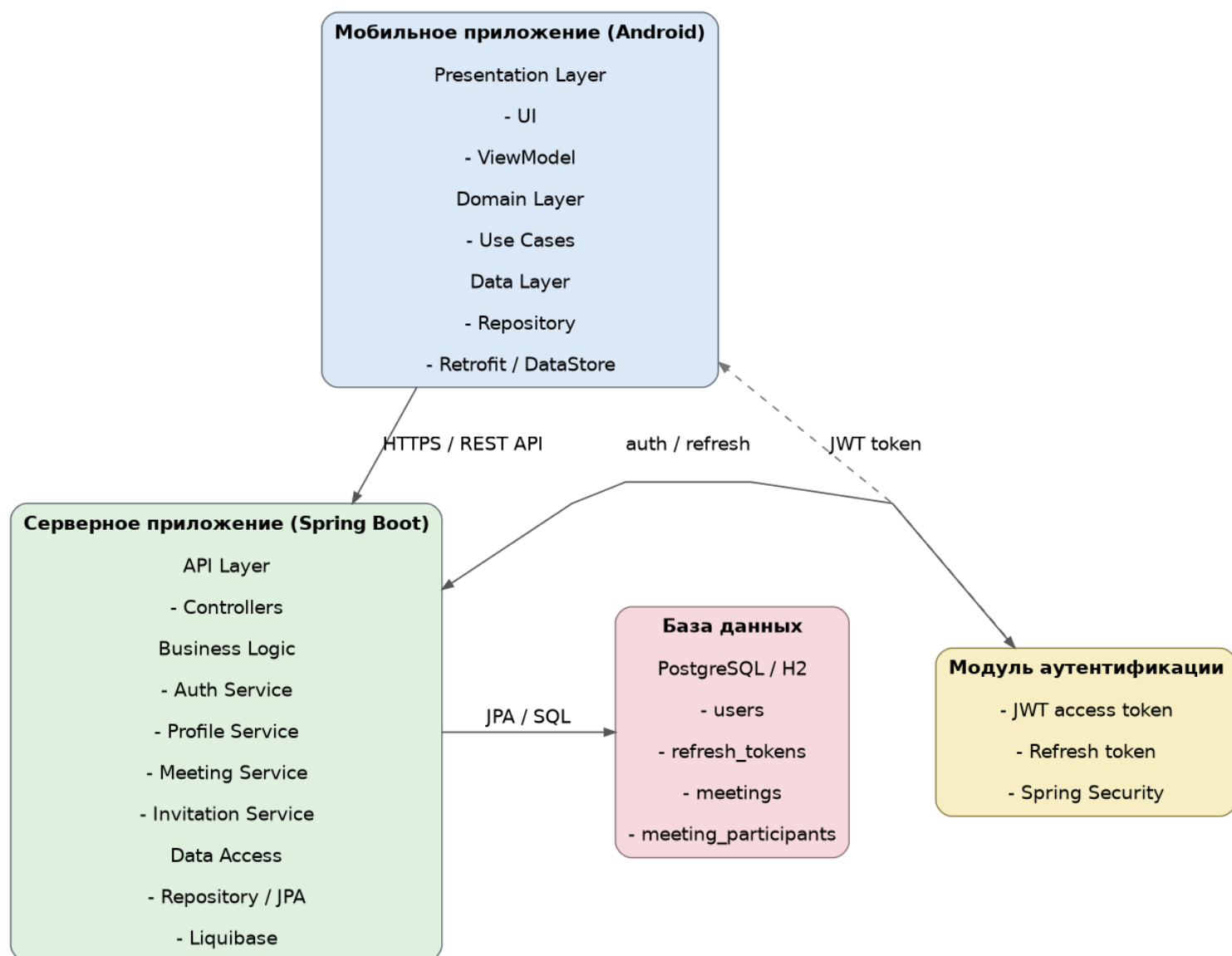


Рисунок 3.2 - Архитектурная диаграмма системы мобильного приложения для покупки билетов на концерты

4.3. Матрица требований

Таблица 1 – Матрица требований системы покупки билетов

Составляющая	Требования	Суть	Компоненты архитектуры
Регистрация и авторизация	Регистрация пользователя	Пользователь должен иметь возможность создать аккаунт с вводом имени, email и пароля. Система должна выполнять валидацию данных и проверку уникальности email.	Уровень представления (экран регистрации), серверная часть (валидация и создание пользователя), уровень данных (таблица users)
	Авторизация пользователя	Пользователь должен иметь возможность войти в систему по email и паролю. После успешной авторизации сервер должен выдать JWT-токен и refresh token для доступа к защищённым ресурсам.	Уровень представления (экран входа), серверная часть (Spring Security, JWT), уровень данных (таблица refresh_tokens)
	Хранение сессии	После успешного входа токен должен сохраняться на клиенте, чтобы пользователь не вводил данные повторно при каждом запуске приложения.	Мобильное приложение (DataStore), уровень представления, REST API
Управление профилем	Просмотр профиля	Авторизованный пользователь должен иметь возможность открыть свой профиль и просмотреть актуальные персональные данные.	Уровень представления (экран профиля), серверная часть (получение данных профиля), уровень данных (users)
	Редактирование профиля	Пользователь должен иметь возможность изменить данные профиля, чтобы другие сотрудники видели актуальную информацию о нём.	UI (форма редактирования), серверная часть (обработка изменений), база данных (users)
	Валидация данных профиля	Система должна проверять корректность введённых пользователем данных при редактировании профиля и возвращать ошибку при некорректном вводе.	Уровень представления, серверная часть, REST API
Создание встречи	Создание встречи	Пользователь должен иметь возможность создать встречу с указанием темы, описания, места, даты, времени начала и окончания.	Уровень представления (экран создания встречи), серверная часть (meeting service), уровень данных (meetings)
	Выбор участников	При создании встречи пользователь должен иметь возможность выбрать других сотрудников из списка пользователей системы и отправить им приглашения.	UI (список сотрудников), серверная часть, таблицы users, meeting_participants
	Почасовые временные слоты	Время встречи должно задаваться только по целым часам. Создание встречи на 9:10, 9:05 и другие дробные значения не допускается.	UI (ограничение выбора времени), серверная бизнес-логика, REST API
	Проверка конфликтов встреч	Перед сохранением встречи система должна убедиться в отсутствии пересечений по времени у организатора и у приглашённых участников.	Серверная часть (бизнес-логика проверки конфликтов), уровень данных (meetings, meeting_participants)
	Сохранение встречи	После успешной проверки система должна сохранить данные о встрече в базе данных.	Серверная часть, Spring Data JPA, таблица meetings

Составляющая	Требования	Суть	Компоненты архитектуры
	Создание приглашений	После сохранения встречи система должна создать записи о приглашениях для всех выбранных участников.	Серверная часть, таблица meeting_participants, уровень данных
Просмотр приглашений	Список активных приглашений	Пользователь должен иметь возможность открыть раздел приглашений и увидеть список актуальных приглашений на встрече.	Уровень представления (экран приглашений), серверная часть, таблица meeting_participants
	Отображение данных приглашения	Каждое приглашение должно содержать тему встречи, время, место проведения и имя организатора.	UI (карточки приглашений), серверная часть, таблицы meetings, users, meeting_participants
Ответ на приглашение	Принятие приглашения	Приглашённый пользователь должен иметь возможность принять приглашение на встречу. После этого статус участия должен быть сохранён в системе, а встреча должна отобразиться в его расписании.	UI (кнопка «Принять»), серверная часть (обновление статуса), таблица meeting_participants
	Отклонение приглашения	Пользователь должен иметь возможность отклонить приглашение. После этого статус участия должен измениться, а встреча не должна включаться в подтверждённое расписание пользователя.	UI (кнопка «Отклонить»), серверная часть, таблица meeting_participants
	Блокировка повторного ответа	После отправки ответа на приглашение кнопки принятия и отклонения должны стать недоступны для повторного изменения, если это предусмотрено логикой системы.	Уровень представления, серверная часть
	Проверка конфликтов при принятии	Если пользователь принимает приглашение, сервер должен дополнительно проверить, не возникает ли конфликт по времени с уже подтверждёнными встречами.	Серверная часть (business logic), уровень данных (meetings, meeting_participants)
Просмотр расписания	Просмотр расписания по дням	Пользователь должен иметь возможность просматривать список своих встреч за выбранный день.	Экран расписания, серверная часть, таблицы meetings, meeting_participants
	Просмотр расписания по неделям	Пользователь должен иметь возможность просматривать встречи за выбранную неделю.	UI (режим «Неделя»), REST API, серверная часть
	Просмотр расписания по месяцам	Пользователь должен иметь возможность просматривать встречи за выбранный месяц.	UI (режим «Месяц»), серверная часть, уровень данных
	Фильтрация расписания	В расписании должны отображаться только встречи, относящиеся к выбранному периоду и подтверждённые для пользователя.	Уровень представления, серверная часть, бизнес-логика выборки
	Просмотр подробной информации о встрече	Пользователь должен иметь возможность открыть встречу из расписания и просмотреть подробную информацию о ней.	UI (экран/диалог деталей встречи), REST API, серверная часть
Безопасность и доступ	Защита API	Все операции, связанные с профилем, созданием встреч, приглашениями и расписанием, должны быть доступны только авторизованным пользователям.	Серверная часть (Spring Security), JWT, REST API

Составляющая	Требования	Суть	Компоненты архитектуры
	Проверка токенов	Сервер должен проверять корректность access token при обращении к защищённым методам API.	Серверная часть (JWT filter, Spring Security)
	Обновление токенов	Система должна поддерживать механизм обновления access token через refresh token без повторного ввода логина и пароля.	Серверная часть, уровень данных (refresh_tokens), мобильное приложение
	Шифрование паролей	Пароли пользователей не должны храниться в открытом виде. В базе должны сохраняться только их хеши.	Серверная часть, BCrypt, таблица users
Данные и БД	Хранение пользователей	Система должна обеспечивать хранение данных о зарегистрированных пользователях.	Уровень данных, таблица users
	Хранение встреч	Система должна обеспечивать хранение всех встреч с данными об организаторе, времени, месте, описании и статусе.	Уровень данных, таблица meetings
	Хранение участников встречи	Система должна хранить список приглашённых пользователей и статус их ответа для каждой встречи.	Уровень данных, таблица meeting_participants
	Хранение refresh token	Система должна хранить refresh token, дату его создания, срок действия и признак отзыва.	Уровень данных, таблица refresh_tokens
Надёжность	Обработка ошибок	При ошибках регистрации, авторизации, редактирования профиля, создания встречи или ответа на приглашение система должна возвращать понятное сообщение без аварийного завершения приложения.	UI (error state), серверная часть (exception handling), REST API
	Целостность данных	Клиентское приложение не должно иметь прямого доступа к базе данных. Все операции должны выполняться только через серверную часть.	Архитектурное разделение: Android-приложение → REST API → БД
	Управляемость структуры БД	Изменения схемы базы данных и начальное заполнение должны выполняться через систему миграций.	Серверная часть, Liquibase, PostgreSQL/H2
Архитектурные требования	Клиент-серверная модель	Система должна быть построена по клиент-серверной архитектуре, где Android-приложение выступает клиентом, а серверное приложение предоставляет API.	Мобильное приложение, Spring Boot, REST API
	Трёхуровневая архитектура	Система должна использовать разделение на уровень представления, уровень бизнес-логики и уровень данных.	Android UI + ViewModel, backend service layer, PostgreSQL/H2
	Разделение клиентских слоёв	Клиентская часть должна быть построена с использованием Clean Architecture и MVVM для удобства сопровождения и тестирования.	Presentation, Domain, Data layers; ViewModel; Use Cases; Repository
	Стандартизованный обмен данными	Обмен данными между клиентом и сервером должен выполняться по HTTP/HTTPS в формате JSON.	Retrofit/OkHttp, REST API, Spring Boot

5. ТЕХНИЧЕСКОЕ ЗАДАНИЕ

5.1. Обоснование выбора ГОСТ для разработки технического задания

Для разработки технического задания (ТЗ) проекта мобильного приложения для планирования встреч, описанного в отчетах №1-4 и №5-8, выбран ГОСТ 34.602-2020 «Техническое задание на создание автоматизированной системы».

Причины выбора:

1. Комплексность проекта:

Проект включает не только разработку клиентского мобильного приложения (Android) и серверной части (REST API на .NET Core), но и взаимодействие с базой данных (PostgreSQL), обеспечение безопасности (шифрование, HTTPS, JWT), а также интеграцию с внешними системами. ГОСТ 34.602-2020 охватывает все аспекты автоматизированных систем (АС), включая требования к структуре, надежности, безопасности, документации и взаимодействию компонентов.

2. Соответствие разделам ТЗ:

Общие сведения: Включает описание системы, участников разработки, сроки и источники финансирования (соответствует разделам отчетов о команде и этапах разработки).

Назначение и цели приложения совпадают с функциональными и нефункциональными требованиями, описанными в матрице требований. Основная цель приложения — упростить процесс планирования, организации и проведения встреч любого формата (деловые встречи, личные встречи, командные созвоны, мероприятия и т.д.).

Требования к системе:

Надежность: Требования к восстановлению данных, обработке ошибок, времени отклика (см. нефункциональные требования).

Безопасность: Шифрование данных, соответствие ФЗ-152, использование HTTPS и JWT.

Эргономика: Требования к интерфейсу (Material Design, адаптивность).

Документирование: Учитывает необходимость подготовки технической и пользовательской документации.

3. Учет автоматизации процессов:

Система автоматизирует процессы поиска встреч, выбора даты и времени, приглашения участников, бронирования ресурсов, а также управление заказами и изменениями, что соответствует определению автоматизированной системы в ГОСТ 34.602-2020.

4. Гибкость стандарта:

ГОСТ 34.602-2020 допускает адаптацию разделов под специфику проекта, что важно для мобильного приложения с клиент-серверной архитектурой. Например, раздел «Требования к видам обеспечения» может быть дополнен описанием использования Kotlin, .NET Core, Retrofit, RxJava и других технологий.

ГОСТ 34.602-2020 выбран как основной стандарт, так как он полностью охватывает требования к автоматизированной системе, включая программные, аппаратные и организационные аспекты.

5.2. Техническое задание на создание автоматизированной системы "Bunch" (по ГОСТ 34.602-2020)

1. Общие сведения

1.1. Наименование системы

-Полное наименование: Мобильное приложение для планирования встреч.

-Краткое наименование: Bunch

1.2. Основание для разработки

Работа выполняется на основании задания по дисциплине «Системная и программная инженерия» от кафедры Математического обеспечения и стандартизации информационных технологий РТУ МИРЭА

1.3. Заказчик и разработчик

Заказчик: РТУ МИРЭА

Разработчик: Студенты группы ИКБО-66-23 (Смирнов А.Ю, Гордильо Ариса, Ковалев А.Э, Тименко И.А.)

1.4. Сроки разработки

Начало работ: 14.02.2026

Окончание работ: 30.05.2026

2. Назначение и цели создания системы

2.1. Назначение

Автоматизация процессов:

- Выбора даты, времени и формата встречи
- Приглашения и согласования участников
- Уведомления пользователей о событиях
- Управления изменениями и отменой встреч

2.2. Цели

- Возможность подтвердить участие во встрече в один клик
- Автоматическое добавление подтверждённой встречи в календаре
- Отправка напоминаний о предстоящей встрече за 1 час и за 1 день

3. Требования к системе

3.1. Функциональные требования

- Регистрация и авторизация пользователя.
- Создание встреч
- Поиск встреч по дате /участникам/названию
- Просмотр информации о встречах
- Редактирование и удаление встреч
- История прошедших встреч
- Уведомления о предстоящих встречах
- Фильтрация и сортировка встреч

3.2. Нефункциональные требования

Производительность:

- Время отклика ≤ 2 сек.
- Поддержка одновременной работы до 10.000 пользователей.

Безопасность:

- Шифрование данных (TLS/SSL).
- Контроль доступа к встречам
- Защита персональных данных

Надежность:

- Резервное копирование данных встреч
- Сохранение данных при сбоях
- Восстановление сессии пользователя

Платформы: Android 12 и выше.

4. Состав и содержание работ

4.1. Анализ требований.

4.2. Прототипирование.

4.3. Разработка:

4.3.1. Клиентская часть (Kotlin, RxJava).

4.3.2. Серверная часть (.NET Core, Entity Framework).

4.4. Тестирование (нагрузочное, юзабилити-тесты).

5. Порядок приёмки

5.1. Этапы тестирования:

5.1.1. Модульное (покрытие кода $\geq 85\%$).

5.1.2. Интеграционное (проверка API).

5.1.3. Приёмочное (соответствие ТЗ).

5.2. Критерии приёмки:

5.2.1. Реализация всех функциональных требований

5.2.2. Корректная работа создания, редактирования и удаления встреч

5.2.3. Успешное прохождение тестов

5.2.4. Стабильная работа приложения

6. Источники разработки

6.1. ГОСТ 34.602-2020.

6.2. Анализ аналогов (приложения Google Calendar, Microsoft Outlook).

Заключение

В ходе выполнения практических работ были завершены этапы проектирования и моделирования системы для планирования встреч.

Разработаны структурные диаграммы (классов, объектов, IDEF0) и информационные модели (DFD), отражающие логику работы системы.

Определена клиент-серверная архитектура с использованием технологий Kotlin (клиент), .NET Core (сервер) и PostgreSQL (СУБД).

Дополнена матрица требований, подтверждающая соответствие функциональных и нефункциональных характеристик проекту.

Полученные результаты являются основой для дальнейшей разработки, тестирования и внедрения системы.